

ARM Interim Checkpoint

Tangina Kader, Amy Showell, Shriya Sistla, Yao Tong

June 4, 2025

1 Group Organisation

The group is organized such that all members claim a similar amount of work, and each member takes ownership of their own emulator tasks. All work is coordinated by communicating progress and requests through a WhatsApp channel.

Our process of dividing tasks involves using a checklist and volunteering by interest, which effectively maintains motivation and dedication. To divide tasks, we first gather in-person to read the specification and create a Google Docs checklist of emulator components. Then, each member of the group volunteers to complete a few tasks they find interesting. Since it is hard to estimate exactly how long each task will take, some leftover tasks are kept open to be claimed as group members complete their initial tasks.

Since no major errors have been merged into the master branch and minimal GitLab technical difficulties have occurred, coordination between members is quite good. When a group member finishes one of their tasks, they typically announce the completed task and ask others to review the merge request, which limits errors. After the request is reviewed and adjustments are made, the code is merged and a reminder to pull is sent on WhatsApp. Lastly, the group member claims an unfinished task from the checklist by writing their name next to an unclaimed task, allowing us to progress quickly while easily tracking development progress. Although each member works independently, sometimes there are sections of code written by others that are relevant yet need to be updated, patched, or clarified. These are also communicated and documented clearly in WhatsApp.

An equal balance of remote and in-person work helps the group function exceptionally well. When working from home, it is easier to concentrate on and complete the task at hand. When working in-person during lab sessions, it is easier to patch bugs and communicate concerns. As a result of the motivation and organization established by the group's workflow, all emulator tasks were completed ahead of the schedule we had designed on day one.

One aspect the group can improve on is slowing down and ensuring everyone truly completes an approximately equal amount of work. Instead of immediately tackling and completing the next open task, it will be beneficial to check on sections where development is slower to identify any issues or roadblocks. This process will ensure all group members keep up-to-date, contribute equally, learn the C language, and maintain communication.

2 Implementation Strategies

Our emulator is split into four main sections: the "hardware" structures, the initialization and fetch cycle, the execution, and the global utility functions. First, the Processor State Register (PSTATE) is represented by a data structure consisting of four Booleans, each representing a PSTATE flag. The state of the ARM is represented by a data structure consisting of an 8-bit integer array representing memory, a 64-bit integer array representing the 31 general-purpose registers, a 64-bit integer representing the PC, and the PSTATE structure. These components are defined in their own `armv8.h` file, and can be updated using the subset of global utility functions found in `modify-regs.c`. Note that the zero register is not included, since it represents the value zero and so can be handled independently.

Next, the ARM memory is initialized using `malloc` and `memset`, setting 2MB of memory to zero before instructions and data are loaded. This is freed at the end of execution. The remaining ARM components are initialized to their defaults as described in the specification file. Afterwards, the fetch cycle repeatedly calls a `decode` function that reads the instruction at the address in the PC, determines the opcode, then passes the instruction to the proper function for execution. Ultimately, `decode` returns a status code indicating successful execution, failed execution, or halt condition reached. This status code also determines if the PC increments, depending on whether a branch has successfully occurred.

The execution of instructions is divided into three modules: Data Processing, Data Transfer, and Branch. These modules are accessed by `decode` depending on bits 28-25 of each instruction. Each module's functions return a status code representing success or failure of execution, which is then propagated by `decode`.

Lastly, the global utility functions are composed of functions that read and write registers, update the PSTATE register, extract bits from instruction fields, and perform sign extensions. These are frequently used across all execution modules, and serve to reduce clutter in the execution modules' code.

Utilities like sign extension, bit extraction, and instruction format data structures can be reused when creating our assembler. Additionally, the emulator directory layout can be reused for our assembler, because the assembler is similar to the emulator in that it performs fetch and encode. For example, the assembler will also have three "encode" modules – Data Processing, Data Transfer, and Branch – to prevent extremely long source files.

One implementation task we will likely find challenging is identifying global variables and utility functions early on. Many of our utility functions were identified and refactored after all emulator functionality was completed, which is time-consuming and inefficient. To mitigate this challenge, group members should ask beforehand if anyone has already implemented a solution for problems that seem very common, such as extracting bits from an instruction or sign-extending an offset. This will speed development by reducing redundancy, shortening source files, and minimizing errors. In the planning stages, the group can also improve in creating a more detailed plan and mapping the code structure. This serves to isolate global variables and utility functions immediately, but also lays out the project directory and clarifies the vision for our end goal.